

Here is the output **before implementing lock**:

```

▼ TERMINAL
ponlork@Ponlorks-MacBook-Air Operating System % python -u "/Users/ponlork/Documents/Operating System/deadlock.py"
VERSION 1: DEADLOCK PROBLEM
=====

Initial state:
  Account 6005: $1000
  Account 6004: $1000

Starting concurrent transfers...

[Thread-1] Attempting transfer $50: 6005 → 6004
[Thread-1] Waiting for lock on Account 6005...

[Thread-2] Attempting transfer $50: 6004 → 6005
[Thread-1] ✓ Acquired lock on Account 6005[Thread-2] Waiting for lock on Account 6004...

[Thread-2] ✓ Acquired lock on Account 6004
[Thread-2] Waiting for lock on Account 6005...
[Thread-1] Waiting for lock on Account 6004...

=====
💀 DEADLOCK DETECTED! 💀
=====

Problem Analysis:
  • Thread-1: Locked 6005, waiting for 6004
  • Thread-2: Locked 6004, waiting for 6005
  • Circular wait = DEADLOCK (deadly embrace)!
=====

```

Here is the output **before implementing lock**:

```

OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  PLAYWRIGHT
> ▼ TERMINAL
ponlork@Ponlorks-MacBook-Air Operating System % python -u "/Users/ponlork/Documents/Operating System/deadlock.py"
[Thread-1] SUCCESS! Transfer completed.
[Thread-1] Released all locks
[Thread-2] ✓ Acquired lock on Account 6004
[Thread-2] Waiting for lock on Account 6005...
[Thread-2] ✓ Acquired lock on Account 6005
[Thread-2] SUCCESS! Transfer completed.
[Thread-2] Released all locks

=====
✓ ALL TRANSFERS COMPLETED SUCCESSFULLY!
=====

Final state:
  Account 6005: $1000
  Account 6004: $1000

No deadlock occurred because both threads always
acquire locks in the same order (6004 before 6005)!
=====

KEY TAKEAWAYS:
=====
1. PROBLEM: Naive locking in arbitrary order → Deadlock
2. SOLUTION: Fixed global lock ordering → No deadlock
3. This implements the 'cooperating processes' approach
   from your slide: establish fixed ordering for resources!
=====

```

<https://github.com/PONLORK19/Operating-System-/tree/Deadlock>